VNF Placement

Programmable Networks A.Y. 2024/25

Outline

- Introduction
- Problem Statement
- Greedy Heuristic

Outline

- Introduction
- Problem Statement
- Greedy Heuristic

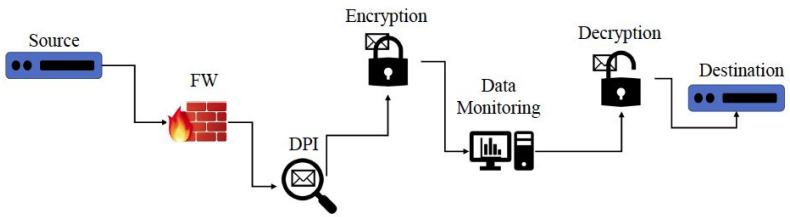
Introduction

- Network Functions (NFs) are the building blocks of Network Services (NSs)
- The relationship between the NS and the NFs composing it is expressed through a graph model, where
  - nodes represent the NFs
  - arcs represent the logical connectivity between to NFs
- In some cases the service can be modeled as an ordered sequence of NFs to be applied on each packet
  - the graph modeling this situation is a chain
    - e.g., a security NS
- More complex NSs are modeled through a meshed graph
  - e.g., the Evolved Packet Core of a 4G network

Introduction

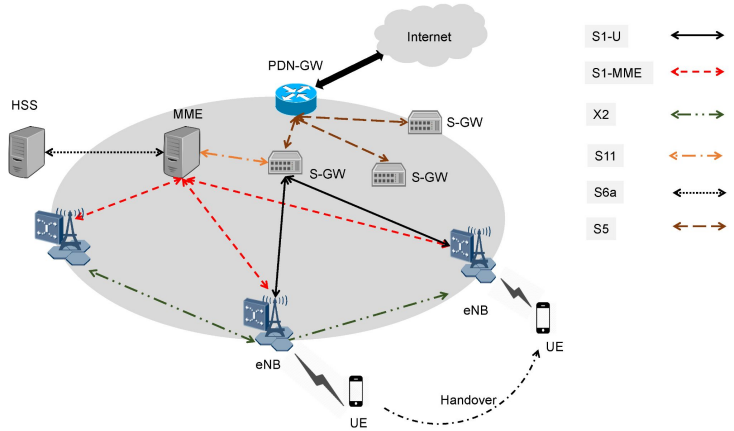
Secure VPN

Example of Network Service modeled by a linear graph



Evolved Packet Core

Example of Network Service modeled by a meshed graph

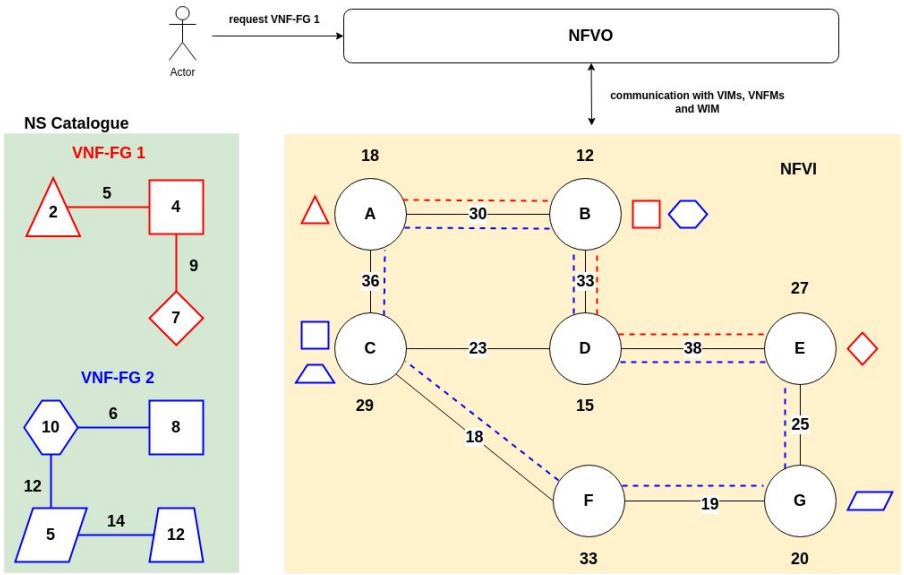


Creating a Network Service

- In case of a Softwarized Network, a NS is represented by a VNF Forwarding Graph
- An instance of VNF-FG is referred to as Network Slice
- To set up a Network Slice requires to perform the following tasks:
  - cloud resources (CPU and storage) must be provisioned to support the creation of VNFs
  - network resources (bandwidth and buffer) must be reserved for each virtual link
  - network switches have to be instructed on how to interconnect the VNFs
- All these activities are performed by the ETSI MANO framework

VNF-FG request

- A user submit its service request in the form of a VNF-FG (among those available in the NS Catalogue of the NFVO)
- NFVO executes an algorithm to asses how to map the VNF-FG in the underlying NFVI
- To do that it uses
  - resource availability information provided by the VIMs
  - It can eventually re-use already running VNF instances
    - VNFMs provide the current status of each VFN instance to the NFVO
- The creation of virtual links is in charge of the WIM



Outline

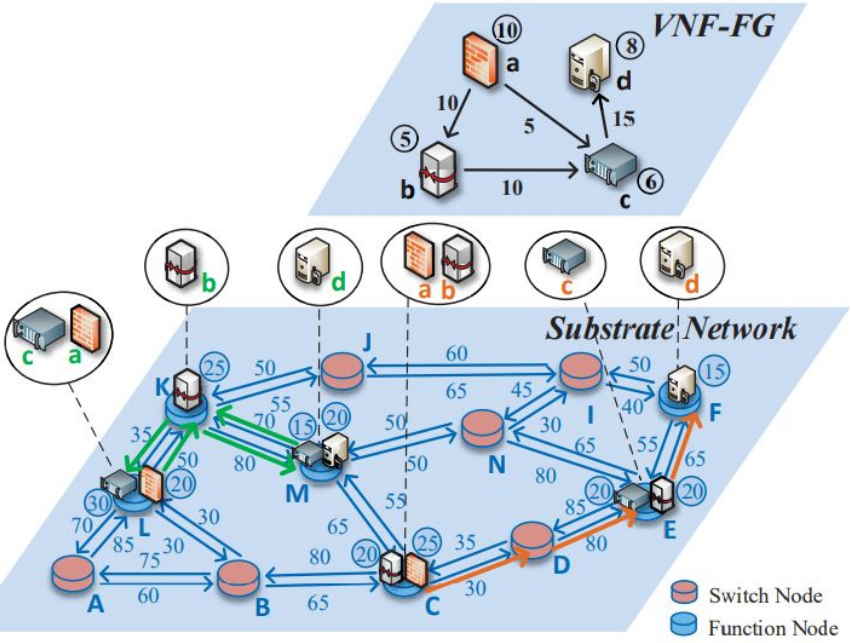
- Introduction
- Problem Statement
- Greedy Heuristic

Overview

- The considered scenario is the one of an NFVIaaS provider that is asked to instantiate a set of Network Slices
- The general problem of allocating cloud and bandwidth resources to perform VNF-FG mapping on the NFVI is known as VNF placement problem
- In the next we consider the **static traffic scenario**, meaning that:
  - bandwidth demand for each request does not change over time
  - each VNF-FG is not changing over time (in terms of VNFs and connections),
  - the number and the set of requested VNF-FGs is known in advance and does not change over time

System Model

VNF-FG	
G^V	VNF-FG.
N^V	Virtual nodes in VNF-FG.
E^V	Virtual links in VNF-FG.
$c_m(n^V)$	Requested resource by node m.
$b_{mn}(e^V)$	Requested resource by link mn.
Substrate Network	
G^S	Substrate network graph.
N^S	Substrate nodes in substrate network.
E^S	Substrate links in substrate network.
$c_i(n^S)$	Available resource of VNF p in function node i.
$b_{ij}(e^S)$	Available resource of link ij.
Variables	
x_i^m	Whether virtual node m is allocated at function node i.
y_{ij}^{mn}	Whether virtual link mn use physical link ij.



Problem Formulation

- The node and link capacity constraints of substrate network are represented by Eq. (1) and Eq. (2)
- Eq. (3) indicates that for a virtual node there must be a VNF in a function node to hold it
- The flow related constraint is ensured in Eq. (4)
- Common objective functions consist in the minimization of: power consumption, rejected requests, etc.

$$\sum_{m \in N^V} c_m(n^V) x_i^m \leq c_i(n^S) \quad \forall i \in N^S$$

$$\sum_{m \in N^V} b_{mn}(e^V) y_{ij}^{mn} \leq b_{ij}(n^S) \quad \forall ij \in E^S$$

$$\sum_{m \in N^V} x_i^m \leq 1 \quad \forall m \in N^V$$

$$\sum_{j \in \delta_i^+} y_{ij}^{mn} - \sum_{j \in \delta_i^-} y_{ji}^{mn} = x_i^m - x_j^n \quad \forall mn \in E^V, i \in N^S$$

Outline

- Introduction
- Problem Statement
- Greedy Heuristic

Accepted VNF-FGs Maximizing VNF Placement Heuristic

- The VNF Placement problem is known to be NP-hard [6], for this reason an heuristic approach is required to allow the NFVO to find a solution in polynomial time
- Greedy heuristic named AVMVPH
- **Two steps**
  - mapping the VNFs leaving the smallest processing capacity available in the NFVI-PoP nodes
  - once the mapping is completed, logical links are mapped in the Substrate Graph by choosing the shortest paths

Heuristic details

- To drive the step 1, the following distance metric is defined

$$distance(VNF_i, N_j) = \begin{cases} (c^i_{VNF} - c^j_{NFVI-PoP})^2 & \text{if } c^i_{VNF} \leq c^j_{NFVI-PoP} \\ Inf & otherwise \end{cases}$$

- c^i_{VNF} is the processing capacity required by the i-th VNF
- $c^j_{NFVI-PoP}$ is the available processing capacity of the j-th NFVI-PoP
- The **best_mapping** tuple is used during the iterative phase of the heuristic
  - `best_mapping.VNF node`- the identity of the considered VNF node
  - `best_mapping.NFVI-PoP node`- the NFVI-PoP node where the VNF is instantiated
  - `best_mapping.value`- the value of the distance among them

Heuristic pseudo-code

Algorithm 1 ACCEPTED VNF-FGS MAXIMIZING VNF PLACEMENT HEURISTIC (AVMVPH)

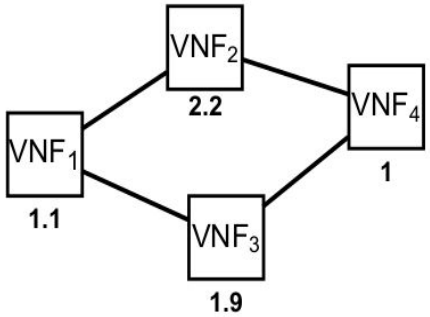
```
1: Input:
   Substrate Graph Ḡ = (V̄, L̄)
   VNF-FG^c graph: G = (V, L)
   M=V: set of VNFs to be mapped
2: while M ≠ ∅ do
3:    best_mapping = (Inf, Null, Null);
4:    for all VNF_i ∈ M do
5:       for all N_c ∈ V̄ do
6:          if distance(VNF_i, N_c) < best_mapping.value then
7:             best_mapping.value = distance(VNF_i, N_c);
8:             best_mapping.VNF_node = VNF_i;
9:             best_mapping.NFVI-PoP_node = N_c;
10:         end if
11:      end for
12:   end for
13:   apply_mapping (best_mapping);
14:   update (M)
15:   update (Ḡ)
16: end while
17: Evaluate the shortest paths which the virtual links are mapped on.
```

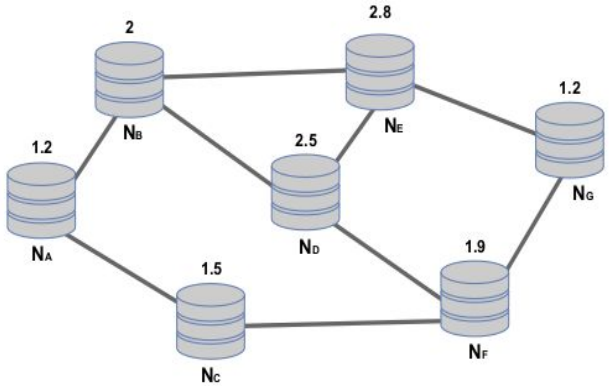
Example

	N_A	N_B	N_C	N_D	N_E	N_F	N_G
VNF_1	0.01	0.81	0.16	1.96	2.89	0.64	0.01
VNF_2	Inf	Inf	Inf	0.09	0.36	Inf	Inf
VNF_3	Inf	0.01	Inf	0.36	0.81	**0**	Inf
VNF_4	0.04	1	0.25	2.25	3.24	0.81	0.04

Distance between the VNFs and compute nodes

$$distance(VNF_i, N_j) = \begin{cases} (c^i_{VNF} - c^j_{NFVI-PoP})^2 & \text{if } c^i_{VNF} \leq c^j_{NFVI-PoP} \\ Inf & otherwise \end{cases}$$



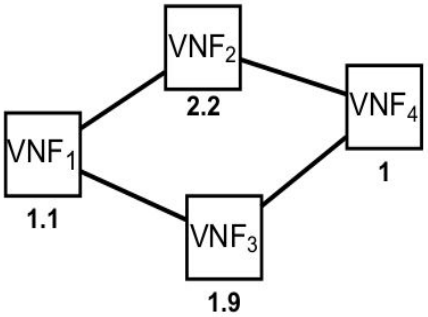


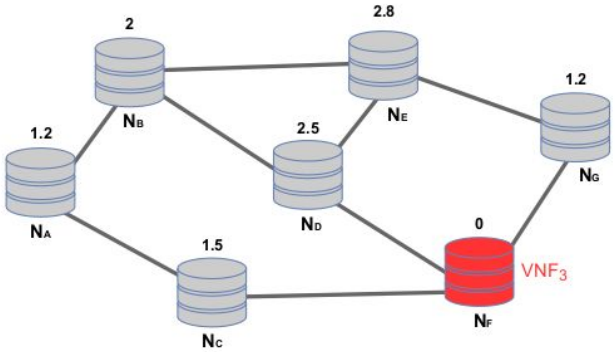
Example

	N_A	N_B	N_C	N_D	N_E	N_F	N_G
VNF_1	**0.01**	0.81	0.16	1.96	2.89	Inf	0.01
VNF_2	Inf	Inf	Inf	0.09	0.36	Inf	Inf
VNF_3							
VNF_4	0.04	1	0.25	2.25	3.24	Inf	0.04

Distance between the VNFs and compute nodes

$$distance(VNF_i, N_j) = \begin{cases} (c^i_{VNF} - c^j_{NFVI-PoP})^2 & \text{if } c^i_{VNF} \leq c^j_{NFVI-PoP} \\ Inf & otherwise \end{cases}$$



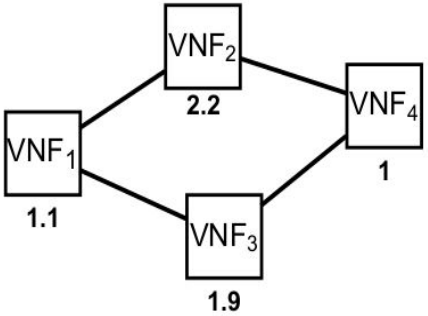


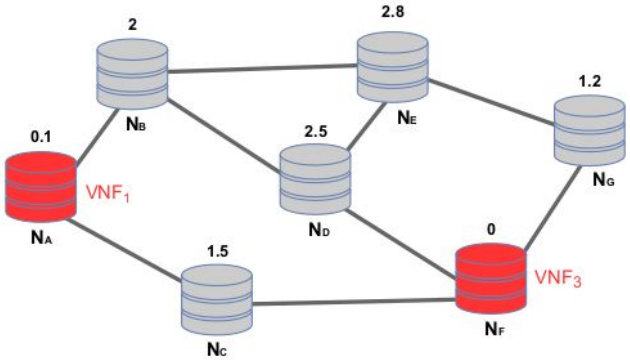
Example

	N_A	N_B	N_C	N_D	N_E	N_F	N_G
VNF_1							
VNF_2	Inf	Inf	Inf	0.09	0.36	Inf	Inf
VNF_3							
VNF_4	Inf	1	0.25	2.25	3.24	Inf	**0.04**

Distance between the VNFs and compute nodes

$$distance(VNF_i, N_j) = \begin{cases} (c^i_{VNF} - c^j_{NFVI-PoP})^2 & \text{if } c^i_{VNF} \leq c^j_{NFVI-PoP} \\ Inf & otherwise \end{cases}$$



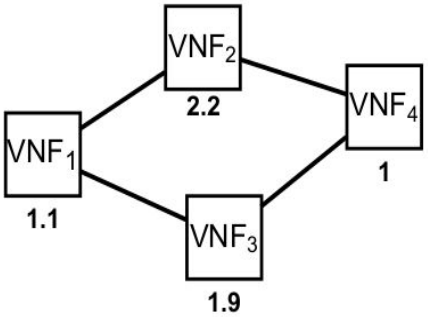


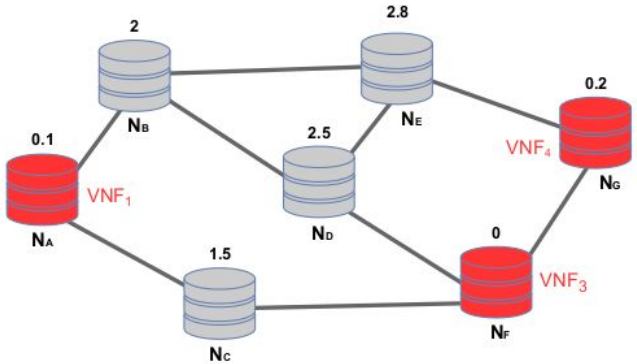
Example

	N_A	N_B	N_C	N_D	N_E	N_F	N_G
VNF_1							
VNF_2	Inf	Inf	Inf	**0.09**	0.36	Inf	Inf
VNF_3							
VNF_4							

Distance between the VNFs and compute nodes

$$distance(VNF_i, N_j) = \begin{cases} (c^i_{VNF} - c^j_{NFVI-PoP})^2 & \text{if } c^i_{VNF} \leq c^j_{NFVI-PoP} \\ Inf & otherwise \end{cases}$$



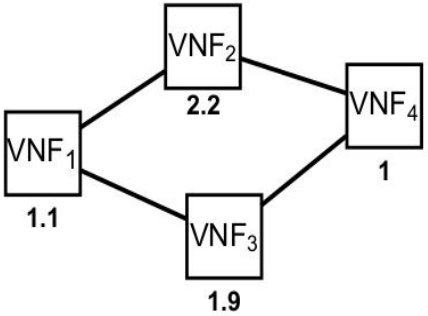


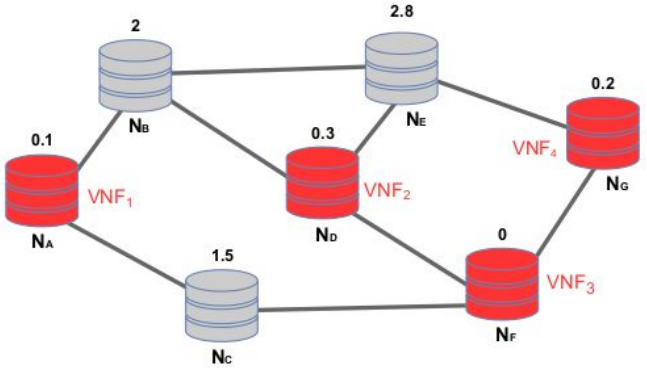
Example

	N_A	N_B	N_C	N_D	N_E	N_F	N_G
VNF_1							
VNF_2							
VNF_3							
VNF_4							

Distance between the VNFs and compute nodes

$$distance(VNF_i, N_j) = \begin{cases} (c^i_{VNF} - c^j_{NFVI-PoP})^2 & \text{if } c^i_{VNF} \leq c^j_{NFVI-PoP} \\ Inf & otherwise \end{cases}$$





Example

- In the last step, each virtual link is mapped over the shortest path in the substrate graph between the VNFs connected by the considered virtual link